

Clase 298 — IA aplicada a la defensa: detección y SOC

Parte: **15 — Seguridad de IA y machine learning** · Fuente: *NIST AI RMF* y literatura de detección de anomalías (Chandola, Banerjee & Kumar) 🕒 Duración estimada: **110 min** · Nivel: **Avanzado**

🎯 Objetivo

Usar la IA como herramienta defensiva de forma realista: detección de anomalías, priorización de alertas, triage y apoyo al analista en el SOC, sin caer en el "AI washing". El alumno construirá un detector de anomalías sobre logs propios, medirá su desempeño con métricas honestas (precisión, recall, falsos positivos) y entenderá su evadibilidad.

📊 Resultados de aprendizaje

Al finalizar, el alumno podrá:

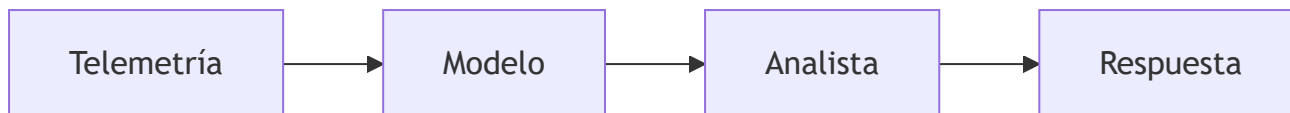
- 1. **Seleccionar** el enfoque adecuado (supervisado, no supervisado, reglas + ML) según el problema del SOC.
- 2. **Construir** un detector de anomalías sobre datos de seguridad propios.
- 3. **Evaluar** el modelo con métricas apropiadas para clases desbalanceadas.
- 4. **Analizar** el coste de los falsos positivos en la carga del analista.
- 5. **Reconocer** la evadibilidad de los detectores por adversarios (ver clase 292).

📖 Temas

#	Tema	Por qué importa
1	Casos de uso reales de IA en el SOC	Dónde ayuda de verdad y dónde no
2	Detección de anomalías no supervisada	No requiere etiquetas de ataques raros
3	Clasificación supervisada de eventos	Cuando hay datos etiquetados de calidad
4	El problema del desbalance y la base rate	Por qué la accuracy engaña
5	Métricas: precisión, recall, F1, FPR	Cómo medir bien la detección
6	Alert fatigue y triage asistido	El coste humano de los falsos positivos
7	Evadibilidad y adversario adaptativo	Los atacantes también atacan al detector

💡 Explicación en profundidad

IA puede priorizar y resumir, pero su salida es evidencia derivada. Se mide con conjuntos representativos, falsos negativos, calibración, deriva y tiempo de analista. Automatizar una decisión exige umbral, revisión y rollback.



El diagrama debe leerse como una cadena de supuestos: verificar un nodo no demuestra los siguientes. La evaluación declara actor, acceso, datos, métrica, umbral y consecuencia; compara una línea base y conserva fallos, no solo promedios.

Caso razonado

El resumen omite un indicador crítico. La interfaz conserva eventos fuente y permite verificar cada afirmación.

Glosario operativo

Término	Definición
Human-in-the-loop	Diseño que asigna revisión humana con autoridad real.
Evaluación	Experimento reproducible ligado a un riesgo y criterio.
Riesgo residual	Exposición que permanece tras controles.

Criterio de dominio

El alumno demuestra dominio cuando formula un escenario específico, reproduce evidencia, explica límites y diseña controles técnicos y de gobierno sin atribuir certeza al modelo.

Definiciones y características

- **Detección de anomalías:** identificar eventos que se desvían del comportamiento normal.
Característica: útil cuando los ataques son raros y no hay etiquetas; propensa a falsos positivos.
- **UEBA (User and Entity Behavior Analytics):** perfilar el comportamiento de usuarios/entidades y alertar desviaciones. *Característica:* requiere baseline y manejo de deriva.
- **Base rate fallacy:** con ataques muy raros, incluso un FPR bajo produce muchísimos falsos positivos.
Característica: invalida la accuracy como métrica.
- **Precisión / Recall:** precisión = de lo alertado, cuánto era real; recall = de lo real, cuánto detectaste.
Característica: hay un compromiso entre ambas.
- **FPR (tasa de falsos positivos):** proporción de eventos benignos marcados como maliciosos; motor del *alert fatigue*.
- **Concept drift:** el comportamiento "normal" cambia con el tiempo; el modelo se degrada si no se reentrena.
- **Evadibilidad:** un atacante puede modelar el detector y camuflar su actividad como normal (adversarial en el dominio de seguridad).

Herramientas y preparación

```
pip install scikit-learn pandas numpy matplotlib
```

- **scikit-learn** (`IsolationForest` , `LocalOutlierFactor`) para anomalías.
- Un dataset de seguridad propio o público bien conocido (p. ej. logs de autenticación sintéticos, o datasets de referencia como los de flujos de red que uses en laboratorio).
- Opcional: un stack SIEM de laboratorio para contextualizar la integración.

Laboratorio guiado

Sobre **datos propios o de laboratorio**.

1. **Define el problema.** Ejemplo: detectar inicios de sesión anómalos a partir de logs (hora, geolocalización, dispositivo, frecuencia).
2. **Explora y normaliza.** Codifica variables categóricas, escala numéricas y separa un baseline de comportamiento "normal".
3. **Entrena un detector no supervisado.**

```
python from sklearn.ensemble import IsolationForest det = IsolationForest(contamination=0.01, random_state=42).fit(X_train) scores = det.decision_function(X_test)
```

4. **Evalúa con métricas honestas.** Si tienes etiquetas, calcula precisión, recall, F1 y FPR; muestra la matriz de confusión. Ignora la accuracy: con 0.1% de ataques, un modelo trivial "todo benigno" tendría 99.9% de accuracy y cero utilidad.
5. **Cuantifica el coste humano.** Estima cuántas alertas/día genera tu FPR sobre el volumen real y cuánto tiempo de analista consume. Ajusta el umbral para equilibrar recall y carga.
6. **Añade contexto (triage).** Enriquez cada alerta con features (reputación de IP, hora, historial del usuario) y ordénalas por score para priorizar.
7. **Prueba la evadibilidad.** Simula un atacante que se mueve "despacio y bajo" imitando patrones normales; observa cómo el recall cae. Concluye que la IA es una capa, no una bala de plata.
8. **Plan de mantenimiento.** Documenta cómo detectarás y corregirás el concept drift (monitorización de distribución, reentrenamiento periódico).

Ejercicios

1. Explica con números por qué la accuracy engaña en detección con clases desbalanceadas.
2. Traza la curva precision-recall de tu detector y elige un umbral operativo justificado.
3. Compara `IsolationForest` con `LocalOutlierFactor` en tu dataset.
4. Estima el coste diario en horas-analista de dos umbrales distintos.
5. Diseña 3 features de contexto que mejorarían el triage.
6. Describe un ataque de evasión realista contra tu detector y una contramedida.

Reto verificable

Entrega un **detector de anomalías evaluado honestamente**: código reproducible, matriz de confusión, curva precision-recall, un umbral operativo justificado por el coste de falsos positivos, y una sección que discuta la evadibilidad.

Criterio de aceptación: el informe NO usa accuracy como métrica principal, justifica el umbral en términos de recall vs. carga de alertas, y contiene al menos un experimento de evasión que muestre la caída del recall. Un revisor debe entender por qué el modelo es útil pero no suficiente por sí solo.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
"Mi detector tiene 99% de accuracy"	Base rate fallacy. Usa precisión, recall, F1 y FPR, no accuracy.
Miles de falsos positivos diarios	Umbral demasiado sensible. Ajusta el umbral y añade contexto para priorizar.
El modelo funcionó y luego dejó de detectar	Concept drift. Monitoriza distribución y reentrena periódicamente.
El atacante pasa desapercibido	Evasión adaptativa. Combina ML con reglas y correlación; no confíes en una sola señal.
Etiquetas de ataque escasas o sucias	Aprendizaje supervisado poco fiable. Considera enfoques no supervisados o semisupervisados.

Preguntas frecuentes

? ¿La IA reemplaza al analista del SOC? No. Reduce ruido y prioriza, pero la decisión y la investigación siguen siendo humanas. Mal calibrada, la IA aumenta la fatiga en vez de reducirla.

? ¿Supervisado o no supervisado para detección? Depende de los datos. Los ataques suelen ser raros y mal etiquetados, así que el no supervisado (anomalías) es común; el supervisado brilla cuando hay etiquetas abundantes y de calidad.

? ¿Por qué tanto énfasis en los falsos positivos? Porque con base rates bajísimas, un FPR pequeño inunda al SOC de alertas inútiles. La precisión operativa manda sobre la accuracy académica.

? ¿Puede un atacante engañar a mi detector? Sí. Los detectores son modelos y por tanto atacables (evasión, envenenamiento del baseline). Diséñalos asumiendo un adversario adaptativo y combínalos con otras defensas.

Referencias

- Chandola, Banerjee & Kumar, "Anomaly Detection: A Survey", ACM Computing Surveys, 2009.
- Sommer & Paxson, "Outside the Closed World: On Using ML for Network Intrusion Detection", IEEE S&P 2010.
- scikit-learn, Outlier detection — https://scikit-learn.org/stable/modules/outlier_detection.html
- NIST AI RMF — <https://www.nist.gov/itl/ai-risk-management-framework>
- MITRE ATLAS — <https://atlas.mitre.org/>

Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.

-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.

Clase anterior

[Clase 297](#) — Seguridad de aplicaciones con LLM: RAG y agentes

Siguiente clase

[Clase 299](#) — IA ofensiva y deepfakes